StorageScope v6.67 — Qualification du véritable Treeview

Auteur : Abdelhalim Talbi

Objectif : comparer la reconstruction intégrale et la réconciliation sur un widget Tkinter réel, et non sur une simulation Python.

Environnement : Linux, serveur Xvfb, trois répétitions par scénario ; valeurs indicatives et non extrapolables à Windows.

Architecture : rapport de rapprochement -> filtrage -> pagination -> reconcile_tree_rows -> widget Treeview.

Changement : lorsque plus de 25 % des positions sont différentes, reconstruction plutôt que déplacements successifs. Le seuil est heuristique et doit être validé sur Windows.

Limites : la mise à jour massive des valeurs reste moins rapide que la reconstruction ; aucune certification de performance Windows ni de compilation native.

Résultats :

1000 lignes / unchanged / rebuild : 2.821 ms (médiane de 3)

1000 lignes / unchanged / reconcile : 2.722 ms (médiane de 3)

1000 lignes / updated / rebuild : 2.931 ms (médiane de 3)

1000 lignes / updated / reconcile : 5.167 ms (médiane de 3)

1000 lignes / reversed / rebuild : 2.942 ms (médiane de 3)

1000 lignes / reversed / reconcile : 5.889 ms (médiane de 3)

Validation : 12 tests ciblés v6.65–v6.67 réussis ; benchmark exécuté sur un vrai Treeview sous Xvfb.

Recette Windows : compiler avec Construire-StorageScope.cmd, lancer l’application, vérifier sélection, ordre, pagination, conflits bloquants, historique et installation/désinstallation.

Exemple fictif (non exécuté) : un opérateur filtre status:blocked sur un inventaire de 100 000 équipements puis consulte les preuves avant de rejeter une proposition.